

Análisis del Comportamiento de E-Converse bajo Diferentes Escenarios de Despliegue

Proyecto Final

Ingeniería de Software III

Universidad Industrial de Santander

Equipo:

Juan Sebastián Otero — 2220053

Daniel Santiago Convers — 2221120

Juan David Paipa — 2220062

Jhon Anderson Vargas — 2220086

Mayo 2026

Índice

1. Introducción	2
2. Descripción de la Aplicación: E-Converse	2
2.1. Funcionalidades principales	2
2.2. Endpoints utilizados para las pruebas de carga	2
3. Estructura del Repositorio	2
4. Cómo Ejecutar el Proyecto	3
4.1. Requisitos previos	3
4.2. Opción A: Ejecución local sin contenedores (desarrollo)	3
4.3. Opción B: Docker Compose (Fase 1 del proyecto)	3
4.4. Opción C: Kubernetes (Fases 2-4)	4
4.5. Poblar la base de datos con 10K productos	4
5. Cómo Visualizar el Notebook	4
5.1. Opción 1: Jupyter Lab/Notebook local	4
5.2. Opción 2: VS Code con extensión Jupyter	4
5.3. Opción 3: Google Colab	4
5.4. Re-ejecutar todas las celdas	5
6. Cómo Reproducir las Pruebas de Carga	5
6.1. Plan de pruebas JMeter (parametrizado)	5
6.2. Ejecutar una prueba puntual desde la línea de comandos	5
6.3. Ejecutar la suite completa (9 escenarios)	5
6.4. Configuración previa requerida (importante)	5
7. Resumen de Resultados	6
8. Enlaces y Recursos	6

1 Introducción

Este documento es el reporte de entrega del proyecto final del curso. Su propósito es servir como guía operativa: explica qué es la aplicación, cómo está organizado el repositorio, cómo levantar el sistema localmente y en Kubernetes, cómo reproducir las pruebas de rendimiento con JMeter y dónde está el análisis completo de los resultados.

El **análisis técnico detallado**, las **tablas** con todas las métricas y las **gráficas comparativas** se encuentran en el notebook Jupyter `Analisis_Rendimiento.ipynb`. Este PDF se enfoca en las instrucciones de ejecución y los enlaces necesarios.

2 Descripción de la Aplicación: E-Converse

E-Converse es una plataforma de comercio electrónico con backend Spring Boot, frontend React y persistencia en MongoDB Atlas.

Componente	Tecnología
Backend	Spring Boot 3.5.6 (Java 20), API REST, JWT
Frontend	React 19 + Vite 7, TailwindCSS, Zustand
Base de Datos	MongoDB Atlas (10,000+ productos)
Contenedorización	Docker (multi-stage)
Orquestación	Kubernetes (K3s v1.35.5)
Pruebas de Carga	Apache JMeter 5.6.3
Análisis	Jupyter Notebook (pandas, matplotlib, seaborn)

2.1 Funcionalidades principales

- Autenticación con JWT (login, registro, roles administrador/cliente).
- Panel administrativo con CRUDs de usuarios, roles, categorías, productos y pedidos.
- Catálogo público con filtros por categoría, precio y nombre.
- Carrito de compras persistente en MongoDB con índice único por usuario.
- Proceso de checkout con simulación de pasarela de pago.
- Historial de pedidos por usuario.

2.2 Endpoints utilizados para las pruebas de carga

Endpoint	Tamaño de respuesta	Característica
GET /api/categorias/list	~800 bytes	Endpoint ligero (4 documentos)
GET /api/productos/list	~3.5 MB	Endpoint pesado (10K documentos)

3 Estructura del Repositorio

```
Proyecto-Spring-Boot-E-converse_vers2/
|-- backend/                               # API Spring Boot (codigo + Dockerfile)
|-- frontend-react/                         # SPA React + Vite (codigo + nginx + Dockerfile)
|-- docker-compose.yml                     # Despliegue local (Fase 1)
|-- k8s/
|   |-- backend-deployment.yaml
|   |-- frontend-deployment.yaml
```

```
|-- jmeter/
|   |-- econverse_load_test.jmx      # Plan JMeter parametrizado
|   |-- run_all_tests.sh             # Script de automatizacion 9
|   escenarios
|   |-- run_1node_only.sh            # Re-ejecucion solo Fase 2
|   |-- results/                     # 9 CSVs (corrida final, 0% errores)
|   |-- results_old/                 # 10 CSVs (corrida inicial, 67%
|   errores)
|-- scripts/
|   |-- populate_db.js               # Insercion de 10K productos
|   |-- setup_roles_admin.js         # Configuracion inicial
|   |-- redeploy-backend.sh          # Re-deploy backend en K3s
|-- Analisis_Rendimiento.ipynb      # Notebook de analisis (PRINCIPAL)
|-- REPORTE_PROYECTO.md              # Diario de problemas y soluciones
|-- Reporte_Final.tex / .pdf         # Este documento
|-- README.md
```

4 Cómo Ejecutar el Proyecto

4.1 Requisitos previos

- Java 17+ y Maven (o usar el contenedor Docker que ya incluye Maven).
- Node.js 18+ y npm (para frontend dev).
- Docker y, opcionalmente, Docker Compose v2.
- Acceso a un clúster Kubernetes (K3s, MicroK8s, Kind o cloud).
- Una URI de MongoDB Atlas o instancia local.

4.2 Opción A: Ejecución local sin contenedores (desarrollo)

```
cd backend
export SPRING_DATA_MONGODB_URI="mongodb+srv://<user>:<pass>@<cluster>/
e_converse"
./mvnw spring-boot:run
# Backend disponible en http://localhost:8080
```

Listing 1: Backend Spring Boot

```
cd frontend-react
npm install
npm run dev
# Frontend disponible en http://localhost:5174
```

Listing 2: Frontend React

4.3 Opción B: Docker Compose (Fase 1 del proyecto)

```
export SPRING_DATA_MONGODB_URI="mongodb+srv://<user>:<pass>@<cluster>/
e_converse"
docker compose up --build
# Frontend en http://localhost:8081
# Backend en http://localhost:8080
```

4.4 Opción C: Kubernetes (Fases 2–4)

1. Construir las imágenes localmente:

```
docker build -t e-converse-backend:latest ./backend
docker build -t e-converse-frontend:latest ./frontend-react
```

2. Distribuir las imágenes a todos los nodos del clúster (importante: *todos* los nodos, no solo los workers):

```
docker save e-converse-backend:latest -o backend.tar
docker save e-converse-frontend:latest -o frontend.tar
# Copiar los tar a cada nodo y luego en cada uno:
sudo k3s ctr images import backend.tar
sudo k3s ctr images import frontend.tar
```

3. Configurar la URI de MongoDB en k8s/backend-deployment.yaml (variable SPRING_DATA_MONGODB_URI)

4. Desplegar:

```
sudo k3s kubectl apply -f k8s/backend-deployment.yaml
sudo k3s kubectl apply -f k8s/frontend-deployment.yaml
sudo k3s kubectl get pods -o wide
# La aplicacion queda en http://<IP-master>:30080
```

4.5 Poblar la base de datos con 10K productos

```
cd scripts
npm install
node setup_roles_admin.js      # Roles y usuario admin
node populate_db.js            # Insercion de 10K productos mock
```

5 Cómo Visualizar el Notebook

El análisis completo, las tablas con todas las métricas y las gráficas comparativas están en Analisis_Rendimiento.ipynb. Hay tres formas de abrirlo:

5.1 Opción 1: Jupyter Lab/Notebook local

```
pip install jupyter pandas matplotlib seaborn
cd /ruta/al/repositorio
jupyter notebook Analisis_Rendimiento.ipynb
# 0 bien:
jupyter lab
```

5.2 Opción 2: VS Code con extensión Jupyter

Abrir el archivo Analisis_Rendimiento.ipynb desde VS Code; la extensión Jupyter detectará el kernel de Python automáticamente.

5.3 Opción 3: Google Colab

Subir el notebook a Google Drive, abrir con Colab y subir manualmente la carpeta `jmeter/results/` a la sesión (o montar Drive). El notebook lee los CSVs desde la ruta relativa `jmeter/results/`.

5.4 Re-ejecutar todas las celdas

Para regenerar todas las gráficas a partir de los CSVs:

```
jupyter nbconvert --to notebook --execute --inplace
Análisis_Rendimiento.ipynb
```

6 Cómo Reproducir las Pruebas de Carga

6.1 Plan de pruebas JMeter (parametrizado)

El archivo `jmeter/econverse_load_test.jmx` es un plan de pruebas con parámetros configurables desde la línea de comandos:

Parámetro	Flag	Valor por defecto
Servidor	-Jserver	10.6.101.163
Puerto	-Jport	30080
Hilos concurrentes	-Jthreads	20
Ramp-up	-Jrampup	20 s
Duración	-Jduration	60 s

6.2 Ejecutar una prueba puntual desde la línea de comandos

```
flatpak run org.apache.jmeter -n \
-t jmeter/econverse_load_test.jmx \
-l jmeter/results/mi_prueba.csv \
-Jserver=10.6.101.163 -Jport=30080 \
-Jthreads=20 -Jrampup=20 -Jduration=60
```

Si JMeter está instalado nativamente, usar `jmeter` en lugar de `flatpak run org.apache.jmeter`.

6.3 Ejecutar la suite completa (9 escenarios)

```
./jmeter/run_all_tests.sh
# Duracion aproximada: 40-50 minutos
# Resultados en jmeter/results/k8s_*.csv
```

El script automatiza todo el ciclo: para cada combinación de nodos $\times$ réplicas, hace `cordon/drain` de los nodos correspondientes, escala el deployment, espera un warmup activo (HTTP 200 consecutivos + 60 s de buffer), corre JMeter y guarda el CSV. Al final restaura el clúster a su estado original.

6.4 Configuración previa requerida (importante)

Antes de correr la suite, verificar que:

1. Todos los nodos tienen la misma imagen del backend. Comparar con:

```
for ip in <ip-master> <ip-worker1> <ip-worker2>; do
  ssh student1@$ip "sudo k3s ctr images ls | grep e-converse-backend | awk '{print \$3}'"
done
```

Los SHAs deben ser idénticos. Si no, exportar desde un nodo correcto e importar en el resto (ver Sección 4 del REPORTE_PROYECTO.md).

2. El clúster está en estado limpio: 3 nodos en Ready, deployments backend/frontend en Running.
3. El endpoint responde 200 antes de empezar:

```
curl -s -o /dev/null -w "%{http_code}\n" http://10.6.101.163:30080/api/categorias/list
```

7 Resumen de Resultados

Para el detalle completo (gráficas comparativas, heatmaps, curvas de escalado, análisis cuantitativo) ver el notebook. Aquí solo el resumen ejecutivo:

Escenario	Nodos	Réplicas	Throughput Cat (req/s)	Latencia Cat (ms)	Error %
1N-1R	1	1	1.37	398	0.0
1N-2R	1	2	1.18	593	0.0
1N-3R	1	3	0.73	1,076	0.0
2N-1R	2	1	1.83	396	0.0
2N-2R	2	2	2.15	318	0.0
2N-3R	2	3	2.60	309	0.0
3N-1R	3	1	1.77	356	0.0
3N-2R	3	2	4.27	294	0.0
3N-3R	3	3	3.27	289	0.0

Configuración óptima: 3 nodos $\times$ 2 réplicas (4.27 req/s, 294 ms de latencia, 0 % errores).

Conclusiones principales:

1. El escalado horizontal funciona, pero tiene un techo definido por la base de datos. Pasar de 2 a 3 réplicas con 3 nodos degrada el throughput (−23 %) por contención de connection pool en MongoDB Atlas.
2. Concentrar la carga en un solo nodo (1N-3R) entrega menos de 1/5 del throughput del mejor escenario.
3. El tamaño del payload (3.5 MB del endpoint de productos) domina sobre la topología: la latencia es 10–15 $\times$ la del endpoint ligero en *todos* los escenarios.

8 Enlaces y Recursos

- Repositorio principal:
https://github.com/JuanSe2731/Proyecto-Spring-Boot-E-converse_vers2
- Notebook de análisis (en el repositorio):
Analisis_Rendimiento.ipynb
- Diario de problemas y soluciones:
REPORTE_PROYECTO.md
- Guía para video explicativo:
GUIA_VIDEO.md
- Documentación Apache JMeter:
<https://jmeter.apache.org/usermanual/get-started.html>

- **Documentación K3s:**
<https://docs.k3s.io/>
- **MongoDB Atlas:**
<https://www.mongodb.com/atlas>